

1. Audio Acquisition

Real-Time Audio Signal Acquisition and Visualization Using MATLAB

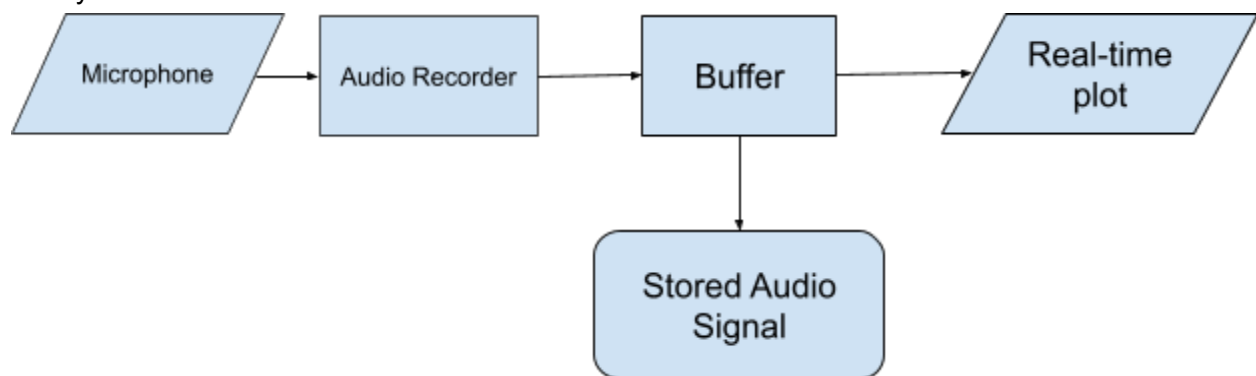
A Real-Time Microphone Interface for Speech Signal Processing

1 – Introduction

Real-time audio acquisition is a fundamental requirement in speech and signal processing systems, including speaker recognition, speech analysis, and human–computer interaction. This document describes the design and implementation of a real-time microphone-based audio signal visualization system using MATLAB.

The implemented system captures speech from a microphone, displays the waveform in real time, and allows the recorded signal to be extracted for further offline analysis. This system serves as the front-end for future speech processing tasks such as framing, feature extraction, and speaker recognition.

2 – System Overview



The system consists of four main components:

- A microphone for capturing speech
- An audio recorder object to digitize the signal
- A timer-based update mechanism to simulate real-time processing
- A waveform plot to visualize the signal continuously

MATLAB's `audiorecorder` and `timer` objects are used to implement asynchronous data acquisition and periodic plotting.

3– Implementation Methodology

3.1 Audio Acquisition

Audio is captured using MATLAB's `audiorecorder` object with a sampling frequency of 16 kHz, 16-bit resolution, and a single channel (mono). The recorder runs asynchronously in the background, allowing continuous data acquisition without blocking program execution.

3.2 Real-Time Plotting Mechanism

Real-time visualization is achieved using a `timer` object configured to execute at fixed intervals. At each timer tick, newly recorded audio samples are retrieved and plotted.

To avoid memory growth, only the most recent fixed-duration segment of the signal (5 seconds) is displayed. This creates a sliding window effect commonly used in real-time signal monitoring systems.

3.3 Synchronization Handling

Since the timer may start before audio samples are available, a synchronization check is performed using the recorder's `TotalSamples` property. This prevents errors related to empty buffers and ensures stable real-time operation.

4 – MATLAB Code Implementation

4.1 Real-Time Audio Visualization Function

The complete MATLAB function used to implement the real-time audio acquisition and visualization system is shown below.

```
function sys = realTime_YW(fs, screen)
% realTime_YW
% Real-time microphone waveform display
%
% Inputs:
%   fs       - Sampling frequency (Hz), default 16000
%   screen    - Time window to display (seconds), default 5
%
% Output:
%   sys.recorder - audiorecorder object
%   sys.clock    - timer object
% -----
% 1. Default arguments
% -----
if nargin < 1
    fs = 16000;
end
if nargin < 2
    screen = 5;
end
% -----
% 2. Create audio recorder
% -----
recObj = audiorecorder(fs, 16, 1); % 16-bit, mono
% -----
% 3. Prepare plot
% -----
bufferLength = fs * screen;
```

```

audioBuffer = zeros(bufferLength,1);
figure('Name','Real-Time Audio');
hPlot = plot(audioBuffer);
ylim([-1 1]);
xlabel('Samples');
ylabel('Amplitude');
title('Microphone Signal');
grid on;
% -----
% 4. Timer for real-time update
% -----
t = timer( ...
    'ExecutionMode','fixedRate', ...
    'Period',0.05, ... % 20 updates per second
    'TimerFcn',@updatePlot);
% -----
% 5. Start recording
% -----
record(recObj);
start(t);
% -----
% 6. Return system handles
% -----
sys.recorder = recObj;
sys.clock = t;
% =====
% Nested function (core logic)
% =====
function updatePlot(~,~)
    % If no samples yet, just wait
    if recObj.TotalSamples == 0
        return;
    end
    data = getaudiodata(recObj);
    % Keep only last "screen" seconds
    if length(data) > bufferLength
        audioBuffer = data(end-bufferLength+1:end);
    else
        audioBuffer = zeros(bufferLength,1);
        audioBuffer(end-length(data)+1:end) = data;
    end
    set(hPlot,'YData',audioBuffer);
    drawnow limitrate;
end
end

```

5 – Execution Procedure

The system was executed using the following procedure:

1. The real-time system was initialized by calling the function with a sampling frequency of 16 kHz and a display window of 5 seconds.
2. Speech was recorded by speaking into the microphone while observing the real-time waveform.
3. The timer was stopped to freeze the waveform display.
4. The audio recorder was stopped, and the recorded signal was extracted for analysis.

```
sys = realTime_YW(16000,5);  
stop(sys.clock);  
stop(sys.recorder);  
s = getaudiodata(sys.recorder);
```

6 – Results and Output

6.1 Real-Time Waveform Display

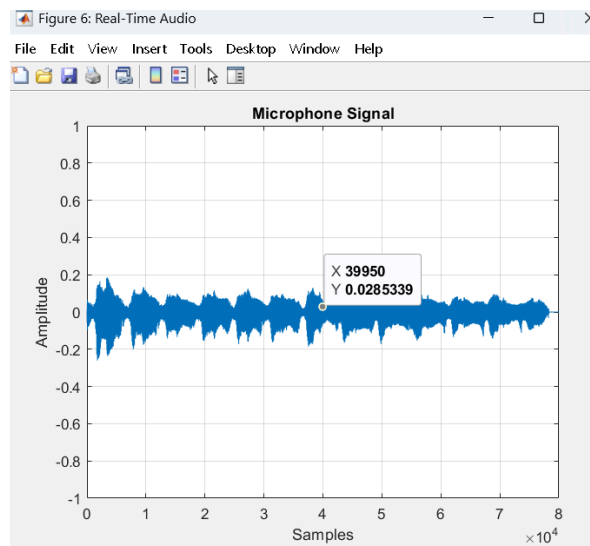


Figure 1: Real-time waveform of speech captured from the microphone.

6.2 Recorded Signal Plot

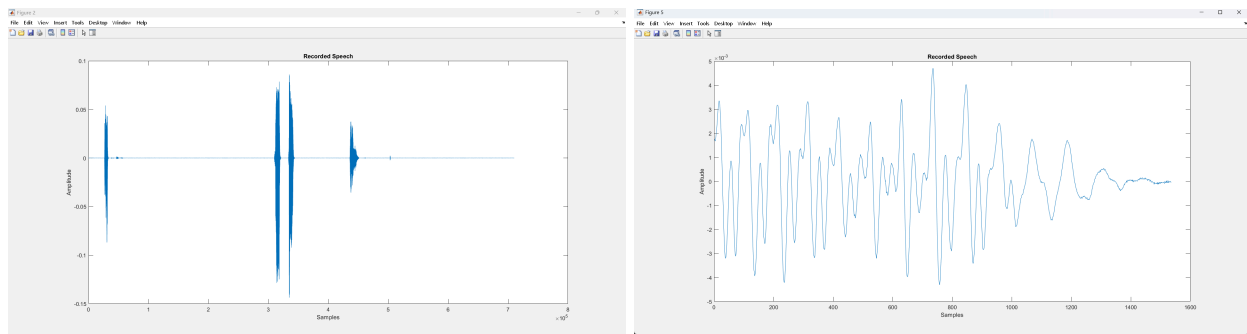


Figure 2: Complete recorded speech signal extracted after stopping the acquisition.

2. Pre-processing

Speech Signal Pre-processing

The recorded speech signal was pre-processed to improve its suitability for feature extraction. First, DC offset removal was performed by subtracting the mean of the signal. This eliminates constant bias introduced by the microphone and recording hardware.

Next, a first-order pre-emphasis filter was applied to amplify high-frequency components of speech. This step compensates for the natural spectral tilt of human speech and enhances important phonetic information. A pre-emphasis coefficient of 0.97 was used, which is commonly adopted in speech processing systems.

Why ?

- Raw speech is messy:
 - low-frequency dominance
 - DC offset
 - Silence
 - channel effects

Pre-processing **cleans the signal** before feature extraction.

- Take your recorded signal `s`
- Produce a new signal `s_pre`

In the preprocessSpeech.m file write the following code:

```
function s_pre = preprocessSpeech(s, alpha)
% preprocessSpeech
% Performs DC removal and pre-emphasis on speech signal
%
% Inputs:
%   s      - raw speech signal (column vector)
%   alpha  - pre-emphasis coefficient (default 0.97)
%
% Output:
%   s_pre  - pre-processed speech signal
if nargin < 2
    alpha = 0.97;
end
% Ensure column vector
s = s(:);
% -----
% 1. DC offset removal
% -----
s_dc = s - mean(s);
% -----
% 2. Pre-emphasis filtering
% -----
s_pre = filter([1 -alpha], 1, s_dc);
end
```


Then run the following,

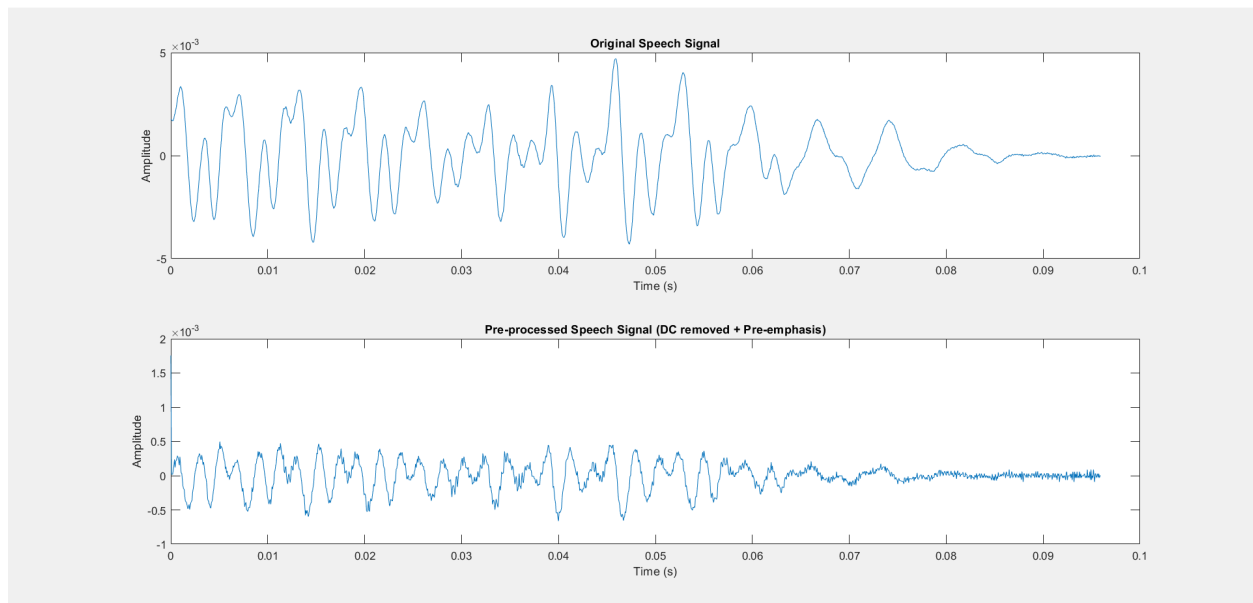
```
s = getaudiodata(sys.recorder);  
fs = 16000;
```

```
s_pre = preprocessSpeech(s);
```

To visualize, use the below code for time-domain comparison

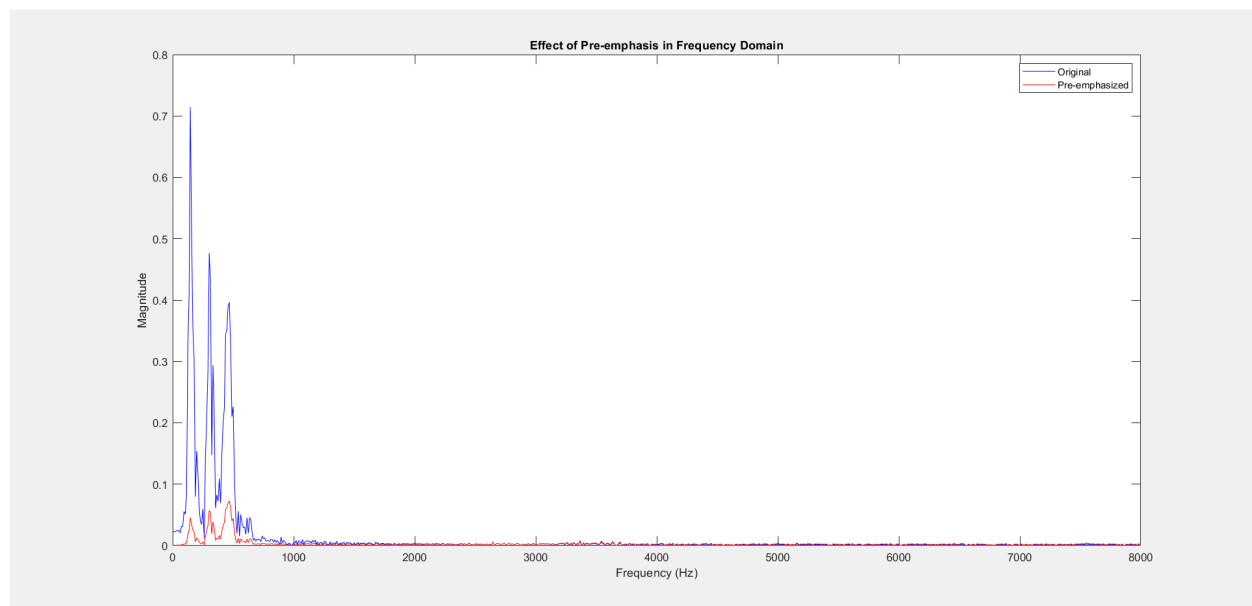
```
t = (0:length(s)-1)/fs;
```

```
figure;  
subplot(2,1,1);  
plot(t, s);  
title('Original Speech Signal');  
xlabel('Time (s)');  
ylabel('Amplitude');  
  
subplot(2,1,2);  
plot(t, s_pre);  
title('Pre-processed Speech Signal (DC removed + Pre-emphasis)');  
xlabel('Time (s)');  
ylabel('Amplitude');
```



For frequency domain proof:

```
N = length(s_pre);  
f = (0:N-1)*(fs/N);  
  
S = abs(fft(s));  
Sp = abs(fft(s_pre));  
  
figure;  
plot(f(1:N/2), S(1:N/2), 'b');  
hold on;  
plot(f(1:N/2), Sp(1:N/2), 'r');  
legend('Original', 'Pre-emphasized');  
xlabel('Frequency (Hz)');  
ylabel('Magnitude');  
title('Effect of Pre-emphasis in Frequency Domain');
```



3. Framing & Windowing (core speech DSP)

Framing and Windowing of Speech Signal

The pre-processed speech signal was segmented into short-time frames to account for the quasi-stationary nature of speech. Each frame had a duration of 25 ms with a frame shift of 10 ms, corresponding to 400 samples per frame and 160 samples overlap at a sampling frequency of 16 kHz.

To reduce spectral leakage and minimize edge discontinuities, a Hamming window was applied to each frame. The resulting windowed frames were arranged as columns in a matrix, forming the basis for subsequent feature extraction.

Why framing is non-negotiable

Speech is **non-stationary** globally but **quasi-stationary** over short intervals.

So we:

- split speech into 20–25 ms frames
- overlap frames by ~10 ms
- apply a Hamming window

This is where speech becomes analyzable.

Frame parameter calculations

Assume:

- Sampling frequency $f_s = 16000$ Hz

Frame length

- 25 ms → standard for speech

$$N = 0.025 \times 16000 = 400 \text{ samples}$$

Frame shift (hop size)

- 10 ms overlap shift

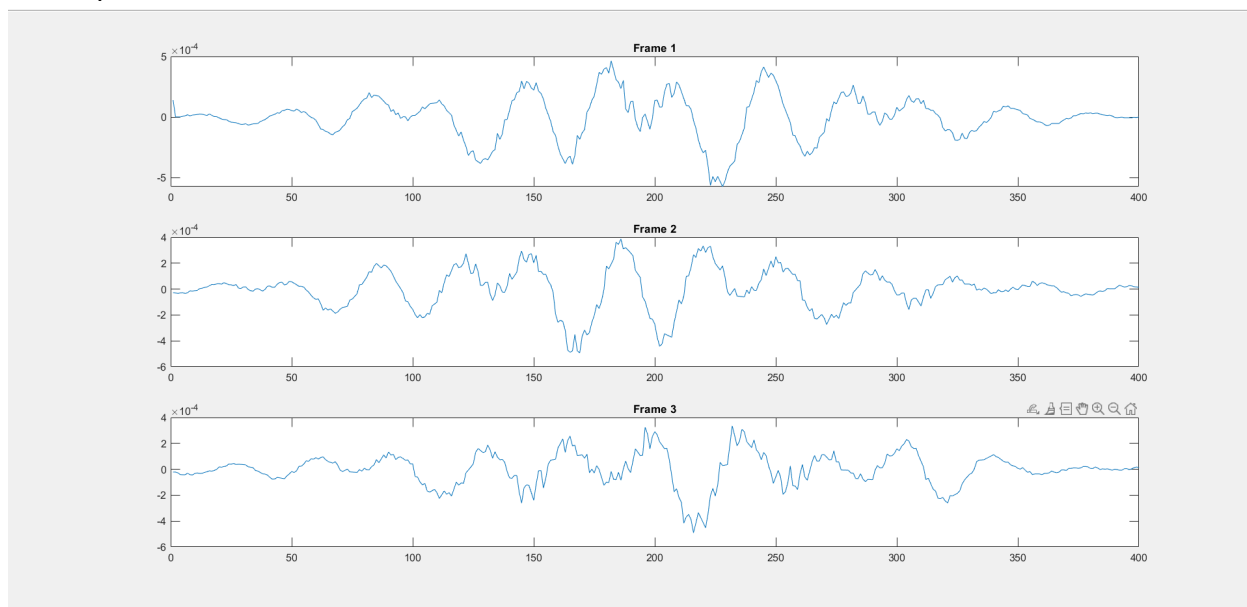
$$M = 0.010 \times 16000 = 160 \text{ samples}$$

Window - Hamming window (industry standard)

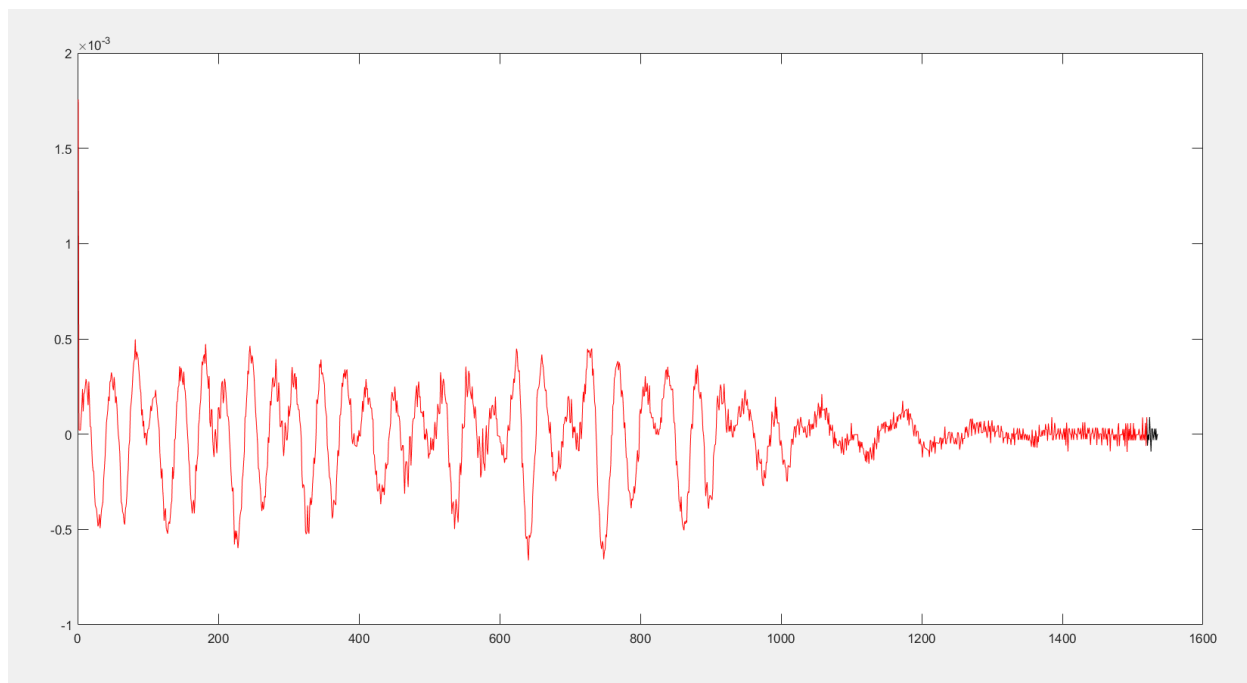
Why Hamming?

- Reduces edge discontinuities
- Improves spectral estimates
- Used in MFCC pipelines

Frame plots



overlap visualization



- frameSpeech.m code

```
function frames = frameSpeech(s, fs, frameLength_ms, frameShift_ms)
% frameSpeech
% Splits speech signal into overlapping, windowed frames
%
% Inputs:
%   s           - pre-processed speech signal
%   fs          - sampling frequency (Hz)
%   frameLength_ms - frame length in milliseconds (e.g., 25)
%   frameShift_ms - frame shift in milliseconds (e.g., 10)
%
% Output:
%   frames      - matrix of frames (each column is one frame)
% Ensure column vector
s = s(:);
% Convert ms to samples
frameLength = round(frameLength_ms * 1e-3 * fs);
frameShift  = round(frameShift_ms * 1e-3 * fs);
signalLength = length(s);
% Number of frames
numFrames = floor((signalLength - frameLength) / frameShift) + 1;
% Pre-allocate frame matrix
frames = zeros(frameLength, numFrames);
% Hamming window
win = hamming(frameLength);
% Frame extraction
for i = 1:numFrames
    startIdx = (i-1)*frameShift + 1;
    endIdx   = startIdx + frameLength - 1;
    frame = s(startIdx:endIdx);
    frames(:,i) = frame .* win;
end
end
```

After that run

```
fs = 16000;
s_pre = preprocessSpeech(s);
frames = frameSpeech(s_pre, fs, 25, 10);
```

frames is a **matrix**

each column = one speech frame

size $\approx 400 \times \text{number_of_frames}$

Verify visually

```
figure;
subplot(3,1,1);
```

```
plot(frames(:,1));  
title('Frame 1');
```

```
subplot(3,1,2);  
plot(frames(:,2));  
title('Frame 2');
```

```
subplot(3,1,3);  
plot(frames(:,3));  
title('Frame 3');
```

Also to visualize overlap

```
figure;  
plot(s_pre,'k');  
hold on;  
  
for i = 1:10  
    startIdx = (i-1)*160 + 1;  
    endIdx = startIdx + 399;  
    plot(startIdx:endIdx, s_pre(startIdx:endIdx),'r');  
end  
  
title('Overlapping Frames on Speech Signal');  
xlabel('Samples');  
ylabel('Amplitude');
```

Raw speech



Pre-processing (DC + pre-emphasis)



Framing (25 ms)



Windowing (Hamming)



Frames matrix  DONE



Feature extraction (MFCC / LPC)

4 – Feature Extraction

4 – Feature Extraction

You must choose one primary feature set.

Recommended path:

- MFCC (Mel-Frequency Cepstral Coefficients)

5 – Speaker Modeling

5 – Speaker Modeling

You need a way to represent each speaker.

Classical & perfect for your project

- **Vector Quantization (VQ)**
- LBG / K-means algorithm
- One codebook per speaker

6 – Speaker Identification (decision logic)

6 – Speaker Identification (decision logic)

What happens here

- Extract features from unknown speaker
- Compute distance to each speaker's codebook
- Choose the speaker with minimum distortion

This is where your system *decides*.

7 – Evaluation

7 – Evaluation

This is where projects win marks.

You test:

- multiple speakers
- multiple test utterances
- compute identification accuracy

Even a simple accuracy table is enough.